

module-1-practical-ml-house-prices

June 28, 2026

```
[1]: """
Danielle Gauthier MSDS 422 Practical ML Module 1
House Prices - Advanced Regression Techniques
"""

# Import modules
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import tree
from sklearn.metrics import accuracy_score
import numpy as np
from scipy.stats import zscore, pearsonr
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Figures inline and set visualization style
%matplotlib inline
sns.set()

# Import test and train datasets
df_train = pd.read_csv('/kaggle/input/
↳house-prices-advanced-regression-techniques/train.csv')
df_test = pd.read_csv('/kaggle/input/
↳house-prices-advanced-regression-techniques/test.csv')

# View first lines of training data
df_train.head(n=4)
```

```
[1]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	\
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	

	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	\
0	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	
1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	

2	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9
3	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2

	YrSold	SaleType	SaleCondition	SalePrice
0	2008	WD	Normal	208500
1	2007	WD	Normal	181500
2	2008	WD	Normal	223500
3	2006	WD	Abnorml	140000

[4 rows x 81 columns]

0.0.1 EDA

The distribution of house prices in the Ames, Iowa sales dataset compiled by Dean De Cock from Kaggle shows that prices are not normally distributed. I used a histogram and a boxplot of **SalePrice** that show distribution of the data to better understand how it was spread. According to the histogram, there is a right skew, with a long right-side tail, indicating most houses are about **\$150,000**, or more expensive. According to the boxplot, most houses are priced between **\$40,000** and **\$340,000**, where there are many outliers for expensive houses.

The indicators with the most missing values are Alley, Masonry veneer type, Fireplace quality, Pool quality, Fence quality, and Miscellaneous features. There are more outliers identified with interquartile range than with z-score. Based on z-score, calculating 3 standard deviations away from the mean, there are 22 outliers. Based on IQR, there are 61 outliers. The lower 25% of houses are mostly from **\$3,875** to **\$129,950**. The top 25% of houses are mostly from **\$214,000** to **\$340,075**. The midspread of the data dispersion is **\$84,050**. It seems IQR is more useful for detecting outliers on this dataset.

I made a correlation heatmap of all indicators in the dataset to choose my predictor variables quickly, and found that the most significant ones were overall quality of the house, above ground living area in the house, and number of cars that fit in the garage. To verify, I plotted scatterplots of these three indicators to see if they had linear relationships. All three indicators have positive relationships, where as they increase in quality or size, the house price increases. For quality, higher quality houses have more variable prices than lower quality. There is a similar variance of ground living area, so that more ground area has more variability of price, decreasing the effect of the predictor. The number of cars that fit in the garage peaks at three cars, but shows the most variability of prices here, and the prices drop significantly for 4 car garage houses. All three indicators are statistically significant with p-values less than 0.05. The highest correlation is house quality at **79.1%**.

I created a new feature by merging the three indicators I mentioned. The Pearson correlation for this new feature was about **8% higher** than the other top indicators alone, at **86.6%**. The p-value for the new feature was **0.0**. This is very statistically significant.

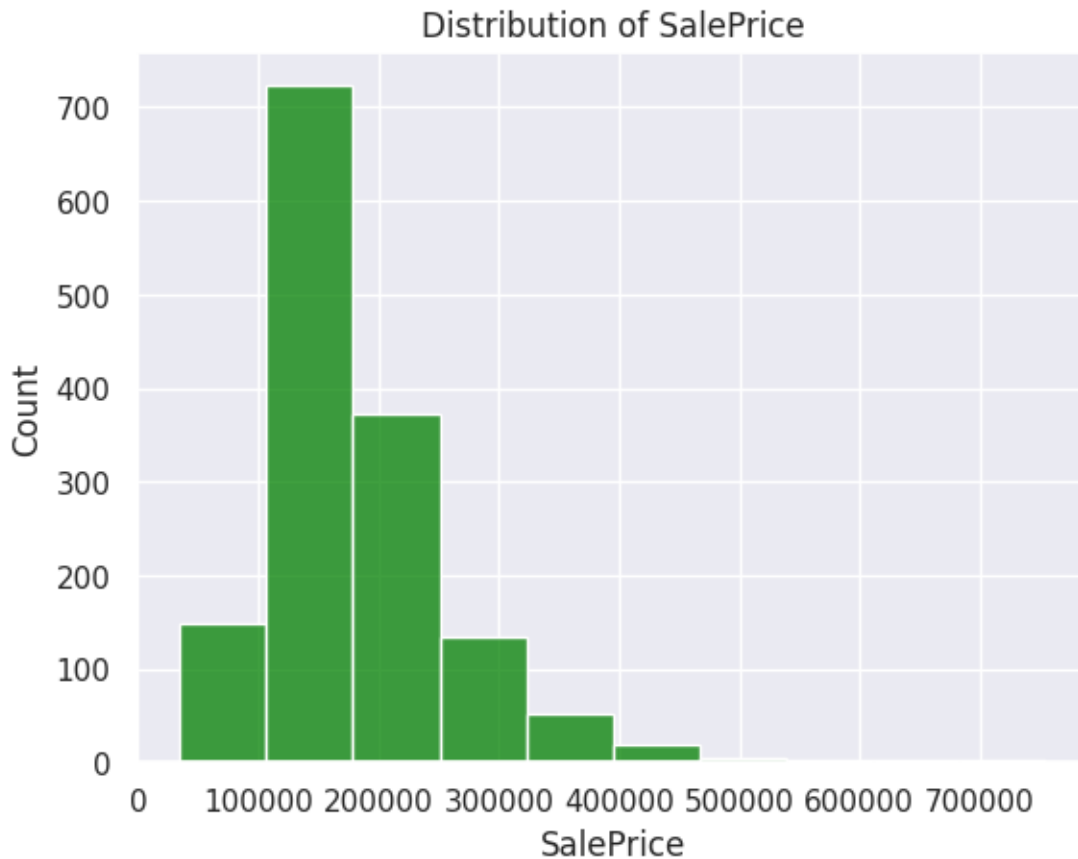
From min-max scaling, we know that the maximum value for price is **\$755,000** and the minimum is **\$34,900**. A house price of **\$500,000**, an outlier from interquartile range, is scaled at only **0.646**. This is another sign of how many outliers there are in the dataset, as in scale this is close to midrange. Using standard scaling on the selected indicators, all means are relatively 0 and represent average, and all variances are relatively 1. This makes the features directly comparable for models that rely on distance, like clustering, or gradient descent, adjusting weights.

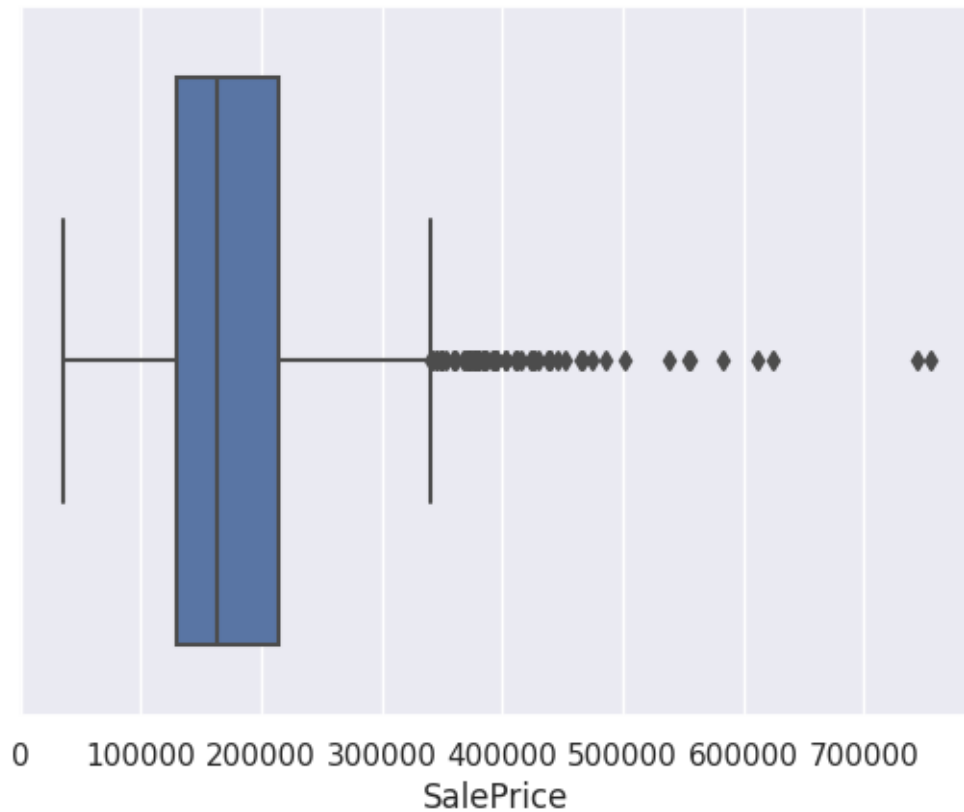
0.0.2 1. Provide appropriate descriptive statistics and visualizations to help understand the marginal distribution of the dependent variable.

```
[2]: # Distribution of SalePrice
sns.histplot(df_train['SalePrice'], bins=10, kde=False, color="green")
plt.title("Distribution of SalePrice")
plt.xlabel("SalePrice")
plt.ylabel("Count")
plt.show()

sns.boxplot(x=df_train["SalePrice"])
plt.show()
```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.
with pd.option_context('mode.use_inf_as_na', True):





0.0.3 2. Investigate missing data and outliers.

```
[3]: # Investigate missing values
df_train.info()
df_train.isnull().sum()

# Investigate outliers with Z score
df_train['Z-score'] = zscore(df_train['SalePrice'])
outliers = df_train[df_train['Z-score'].abs() > 3]
outliers_z_score = (df_train['Z-score'].abs() > 3).sum()
print("Number of outliers:", outliers_z_score)

plt.figure(figsize=(10, 6))

plt.scatter(df_train['SalePrice'].index, df_train['Z-score'], label='Price_
↳Points')
plt.scatter(outliers['SalePrice'].index, outliers['Z-score'], color='red',
↳label='Outlier')

plt.xlabel('House Value')
```

```

plt.ylabel('Z-score')
plt.title('Scatter Plot of House Value vs. Z-score')
plt.legend()
plt.grid(True)
plt.show()

# Investigate outliers with IQR, in case of skew
SalePrice = df_train["SalePrice"].dropna()
Q1 = np.percentile(SalePrice, 25, interpolation = 'midpoint')
Q2 = np.percentile(SalePrice, 50, interpolation = 'midpoint')
Q3 = np.percentile(SalePrice, 75, interpolation = 'midpoint')

print('Q1 25 percentile of the given data is, ', Q1)
print('Q1 50 percentile of the given data is, ', Q2)
print('Q1 75 percentile of the given data is, ', Q3)

IQR = Q3 - Q1
print('Interquartile range is', IQR)

low_lim = Q1 - 1.5 * IQR
up_lim = Q3 + 1.5 * IQR
print('low_limit is', low_lim)
print('up_limit is', up_lim)

outlier = []
for x in SalePrice:
    if ((x > up_lim) or (x < low_lim)):
        outlier.append(x)
print('Outlier in the dataset is', outlier)
print("Number of outliers:", len(outlier))

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1460 entries, 0 to 1459
```

```
Data columns (total 81 columns):
```

#	Column	Non-Null Count	Dtype
0	Id	1460 non-null	int64
1	MSSubClass	1460 non-null	int64
2	MSZoning	1460 non-null	object
3	LotFrontage	1201 non-null	float64
4	LotArea	1460 non-null	int64
5	Street	1460 non-null	object
6	Alley	91 non-null	object
7	LotShape	1460 non-null	object
8	LandContour	1460 non-null	object
9	Utilities	1460 non-null	object
10	LotConfig	1460 non-null	object
11	LandSlope	1460 non-null	object

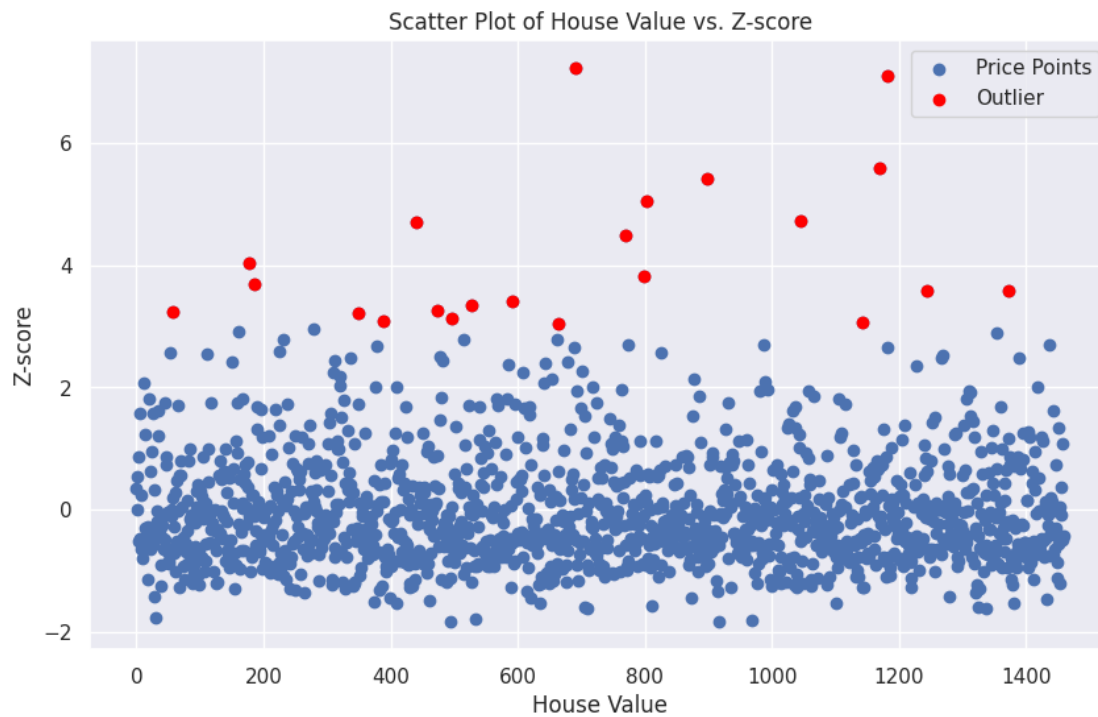
12	Neighborhood	1460	non-null	object
13	Condition1	1460	non-null	object
14	Condition2	1460	non-null	object
15	BldgType	1460	non-null	object
16	HouseStyle	1460	non-null	object
17	OverallQual	1460	non-null	int64
18	OverallCond	1460	non-null	int64
19	YearBuilt	1460	non-null	int64
20	YearRemodAdd	1460	non-null	int64
21	RoofStyle	1460	non-null	object
22	RoofMatl	1460	non-null	object
23	Exterior1st	1460	non-null	object
24	Exterior2nd	1460	non-null	object
25	MasVnrType	588	non-null	object
26	MasVnrArea	1452	non-null	float64
27	ExterQual	1460	non-null	object
28	ExterCond	1460	non-null	object
29	Foundation	1460	non-null	object
30	BsmtQual	1423	non-null	object
31	BsmtCond	1423	non-null	object
32	BsmtExposure	1422	non-null	object
33	BsmtFinType1	1423	non-null	object
34	BsmtFinSF1	1460	non-null	int64
35	BsmtFinType2	1422	non-null	object
36	BsmtFinSF2	1460	non-null	int64
37	BsmtUnfSF	1460	non-null	int64
38	TotalBsmtSF	1460	non-null	int64
39	Heating	1460	non-null	object
40	HeatingQC	1460	non-null	object
41	CentralAir	1460	non-null	object
42	Electrical	1459	non-null	object
43	1stFlrSF	1460	non-null	int64
44	2ndFlrSF	1460	non-null	int64
45	LowQualFinSF	1460	non-null	int64
46	GrLivArea	1460	non-null	int64
47	BsmtFullBath	1460	non-null	int64
48	BsmtHalfBath	1460	non-null	int64
49	FullBath	1460	non-null	int64
50	HalfBath	1460	non-null	int64
51	BedroomAbvGr	1460	non-null	int64
52	KitchenAbvGr	1460	non-null	int64
53	KitchenQual	1460	non-null	object
54	TotRmsAbvGrd	1460	non-null	int64
55	Functional	1460	non-null	object
56	Fireplaces	1460	non-null	int64
57	FireplaceQu	770	non-null	object
58	GarageType	1379	non-null	object
59	GarageYrBlt	1379	non-null	float64

60	GarageFinish	1379	non-null	object
61	GarageCars	1460	non-null	int64
62	GarageArea	1460	non-null	int64
63	GarageQual	1379	non-null	object
64	GarageCond	1379	non-null	object
65	PavedDrive	1460	non-null	object
66	WoodDeckSF	1460	non-null	int64
67	OpenPorchSF	1460	non-null	int64
68	EnclosedPorch	1460	non-null	int64
69	3SsnPorch	1460	non-null	int64
70	ScreenPorch	1460	non-null	int64
71	PoolArea	1460	non-null	int64
72	PoolQC	7	non-null	object
73	Fence	281	non-null	object
74	MiscFeature	54	non-null	object
75	MiscVal	1460	non-null	int64
76	MoSold	1460	non-null	int64
77	YrSold	1460	non-null	int64
78	SaleType	1460	non-null	object
79	SaleCondition	1460	non-null	object
80	SalePrice	1460	non-null	int64

dtypes: float64(3), int64(35), object(43)

memory usage: 924.0+ KB

Number of outliers: 22



```

Q1 25 percentile of the given data is, 129950.0
Q1 50 percentile of the given data is, 163000.0
Q1 75 percentile of the given data is, 214000.0
Interquartile range is 84050.0
low_limit is 3875.0
up_limit is 340075.0
Outlier in the dataset is [345000, 385000, 438780, 383970, 372402, 412500,
501837, 475000, 386250, 403000, 415298, 360000, 375000, 342643, 354000, 377426,
437154, 394432, 426000, 555000, 440000, 380000, 374000, 430000, 402861, 446261,
369900, 451950, 359100, 345000, 370878, 350000, 402000, 423000, 372500, 392000,
755000, 361919, 341000, 538000, 395000, 485000, 582933, 385000, 350000, 611657,
395192, 348000, 556581, 424870, 625000, 392500, 745000, 367294, 465000, 378500,
381000, 410000, 466500, 377500, 394617]
Number of outliers: 61

```

0.0.4 3. Investigate at least three potential predictors of the dependent variable and provide appropriate graphs / statistics to demonstrate the relationships.

```

[4]: # Heatmap of indicators
plt.figure(figsize=(10,8))
corr = df_train.corr(numeric_only=True)
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
plt.title("Correlation Heatmap")
plt.show()

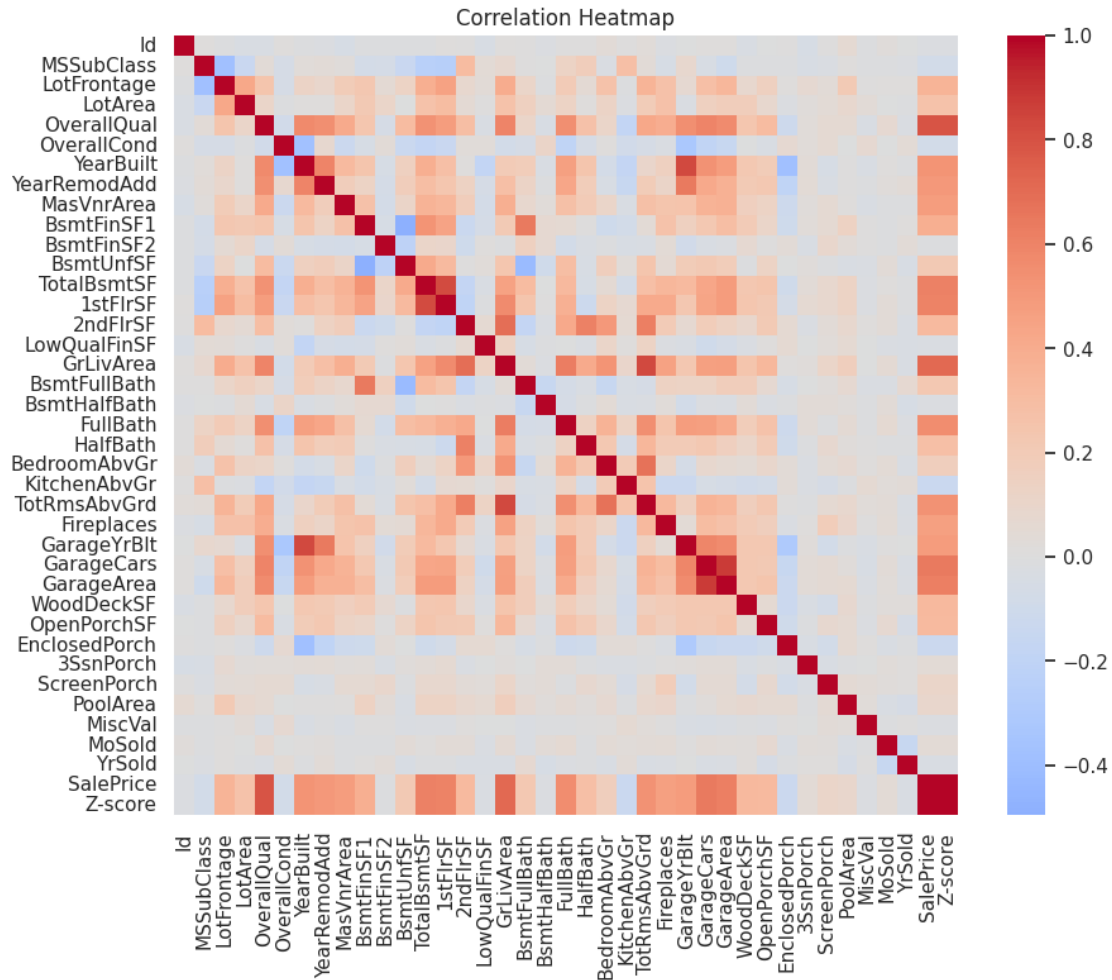
# Scatterplots to show relationships and P values of chosen indicators
sns.scatterplot(data=df_train, x="OverallQual", y="SalePrice")
plt.show()
corr, p_value = pearsonr(df_train["OverallQual"], df_train["SalePrice"])
print("Pearson correlation of Overall House Quality to Sale Price:", corr,
      ↪ "p-value:", p_value)
sns.scatterplot(data=df_train, x="GrLivArea", y="SalePrice")
plt.show()
corr, p_value = pearsonr(df_train["GrLivArea"], df_train["SalePrice"])
print("Pearson correlation of Above Ground Area to Sale Price:", corr, "p-value:
      ↪", p_value)
sns.scatterplot(data=df_train, x="GarageCars", y="SalePrice")
plt.show()
corr, p_value = pearsonr(df_train["GarageCars"], df_train["SalePrice"])
print("Pearson correlation of No. Cars in Garage to Sale Price:", corr,
      ↪ "p-value:", p_value)

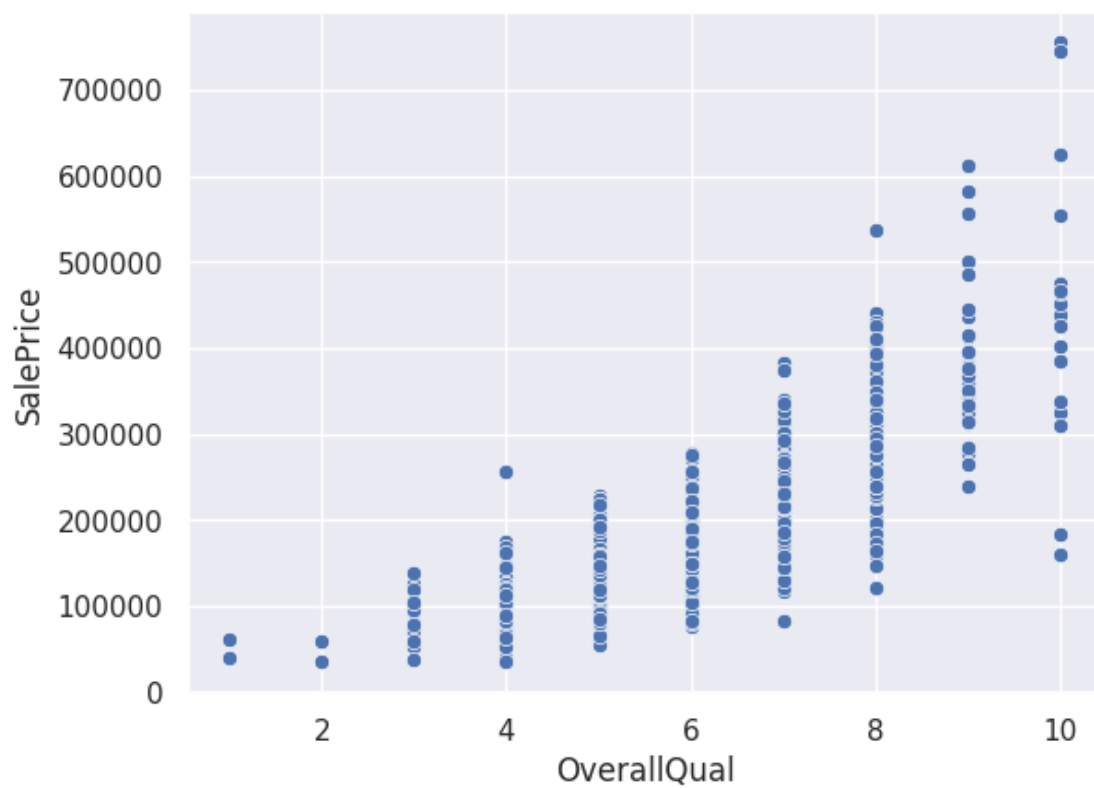
```

```

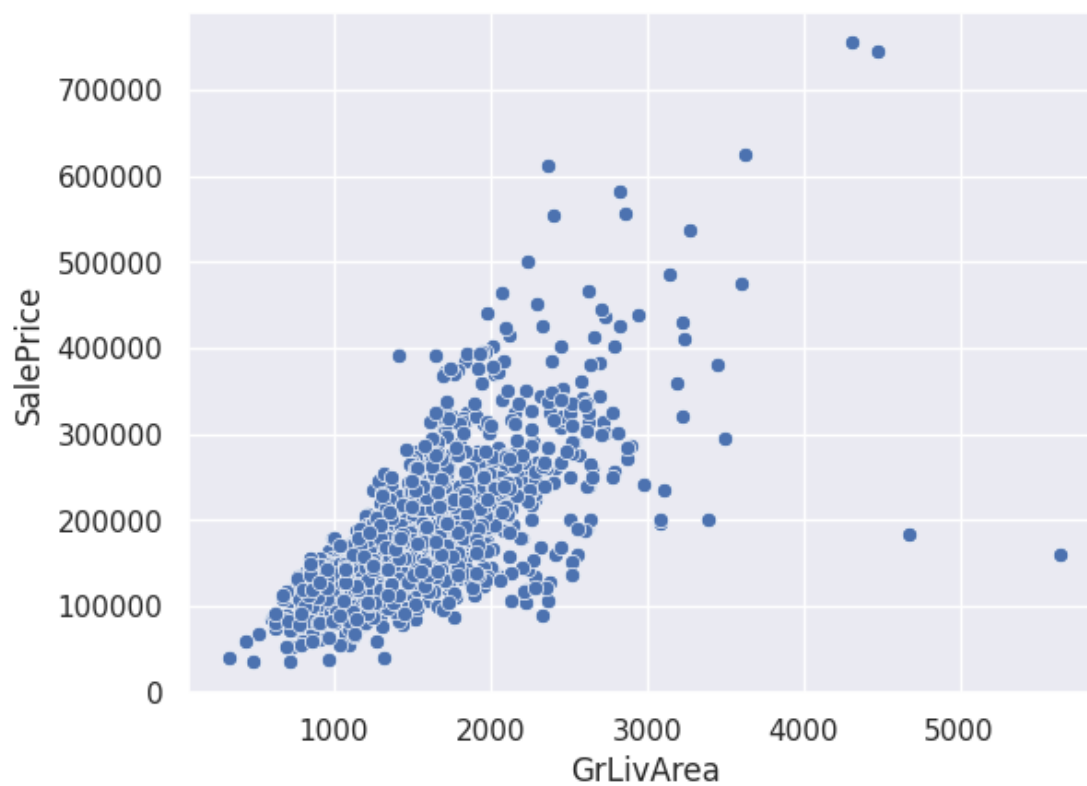
/usr/local/lib/python3.11/dist-packages/matplotlib/colors.py:721:
RuntimeWarning: invalid value encountered in less
  xa[xa < 0] = -1

```

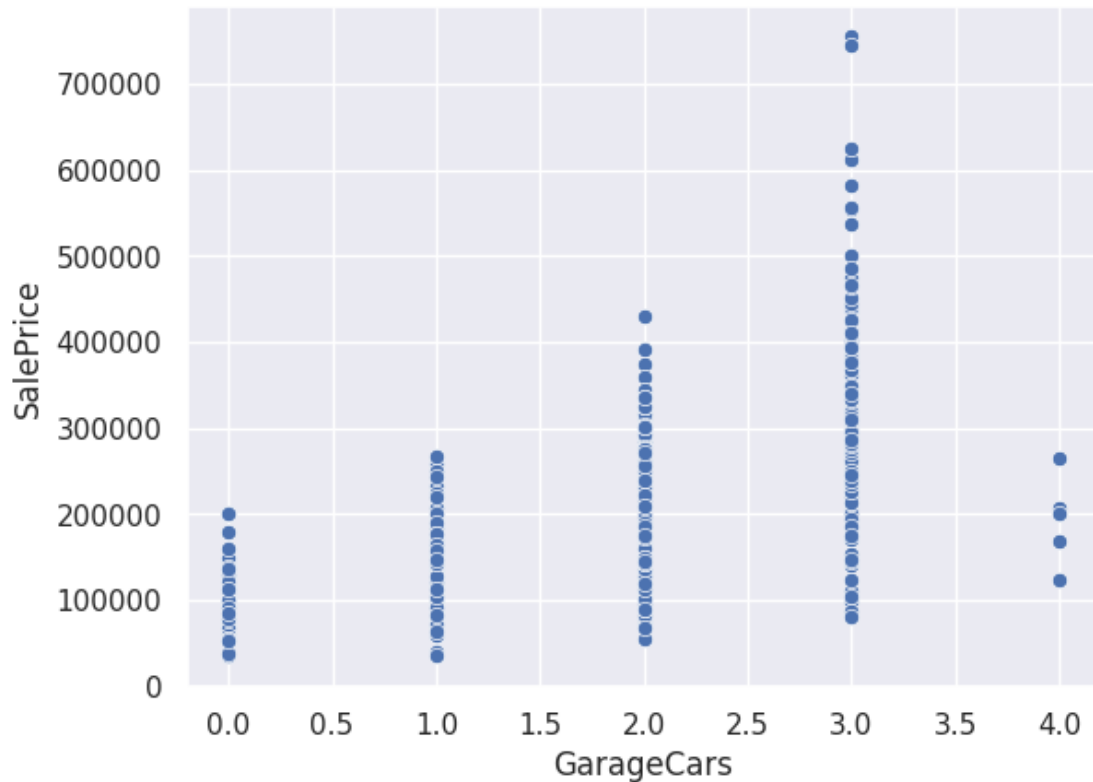





Pearson correlation of Overall House Quality to Sale Price: 0.7909816005838051
p-value: 2.185675268e-313



Pearson correlation of Above Ground Area to Sale Price: 0.7086244776126521
p-value: 4.518033646778376e-223



Pearson correlation of No. Cars in Garage to Sale Price: 0.6404091972583521
p-value: 2.4986441671772544e-169

0.0.5 4. Engage in feature creation by splitting, merging, or otherwise generating a new predictor.

```
[5]: # Feature creation
# Select relevant features
selected_data = df_train[["OverallQual", "GrLivArea", "GarageCars",
    ↳ "SalePrice"]]
extracted_data = selected_data.copy()

# Create a toy feature: predicted price as product of main features
extracted_data["predicted_price"] = (
    extracted_data["OverallQual"] *
    extracted_data["GrLivArea"] *
    extracted_data["GarageCars"]
)

# Create a relative efficiency feature: how close actual price is to predicted
extracted_data["efficiency"] = extracted_data["SalePrice"] /
    ↳ extracted_data["predicted_price"]
```

```

# Display sample
print(extracted_data.head())

# Correlation value
corr, p_value = pearsonr(extracted_data["predicted_price"],
    ↪ extracted_data["SalePrice"])
print("Pearson correlation of Predicted Price to Actual Sale Price:", corr,
    ↪ "p-value:", p_value)

```

	OverallQual	GrLivArea	GarageCars	SalePrice	predicted_price	efficiency
0	7	1710	2	208500	23940	8.709273
1	6	1262	2	181500	15144	11.984945
2	7	1786	2	223500	25004	8.938570
3	7	1717	3	140000	36057	3.882741
4	8	2198	3	250000	52752	4.739157

Pearson correlation of Predicted Price to Actual Sale Price: 0.866488951812828
p-value: 0.0

0.0.6 5. Using the dependent variable, perform both min-max and standard scaling in Python.

```

[6]: # Min-max scaling
scaler = MinMaxScaler()

# Fit and transform SalePrice column once
df_train['SalePrice_minmax'] = scaler.fit_transform(df_train[['SalePrice']])

# Show some results
print("Raw SalePrice Min:", scaler.data_min_)
print("Raw SalePrice Max:", scaler.data_max_)
print(df_train[['SalePrice', 'SalePrice_minmax']].head())

# Example: scale a single new value
print(scaler.transform([[500000]]))

# Log transformation
df_train['SalePrice_log'] = np.log1p(df_train['SalePrice'])

# Standard scaling on predictors
selected_data = ["OverallQual", "GrLivArea", "GarageCars"]
sc = StandardScaler()

# Fit and transform training predictors
df_train[selected_data] = sc.fit_transform(df_train[selected_data])

# Print mean and variance of transformed predictors

```

```
print("Means after log and scaling predictors:\n", df_train[selected_data].
      ↪mean())
print("\nVariances after log and scaling predictors:\n",
      ↪df_train[selected_data].var())
```

Raw SalePrice Min: [34900.]

Raw SalePrice Max: [755000.]

	SalePrice	SalePrice_minmax
0	208500	0.241078
1	181500	0.203583
2	223500	0.261908
3	140000	0.145952
4	250000	0.298709

[[0.64588252]]

Means after log and scaling predictors:

OverallQual	1.289684e-16
GrLivArea	-1.314017e-16
GarageCars	1.095014e-16

dtype: float64

Variances after log and scaling predictors:

OverallQual	1.000685
GrLivArea	1.000685
GarageCars	1.000685

dtype: float64

/usr/local/lib/python3.11/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but MinMaxScaler was fitted with feature names
warnings.warn(

0.0.7 Create submission data

```
[7]: # Feature creation on test data
test_data = df_test[["Id", "OverallQual", "GrLivArea", "GarageCars"]].copy()

test_data = test_data.fillna(0)

# Compute predicted SalePrice
test_data["SalePrice"] = (
    test_data["OverallQual"].astype(float) *
    test_data["GrLivArea"].astype(float) *
    test_data["GarageCars"].astype(float)
)

# Ensure no NaNs or non-numeric
assert test_data["SalePrice"].notnull().all(), "NaNs still present in SalePrice!"
      ↪"
```

```
output = test_data[["Id", "SalePrice"]]  
  
# Save to CSV  
output.to_csv("/kaggle/working/predicted_prices.csv", index=False)  
print(output.head())
```

	Id	SalePrice
0	1461	4480.0
1	1462	7974.0
2	1463	16290.0
3	1464	19248.0
4	1465	20480.0